

实验二 决策树算法的设计与实现

一、实验目的

1. 了解分类决策树基本原理，掌握决策树分类器方法；
2. 编程实现决策树分类器；
3. 使用决策树分类器实现分类，可视化决策树分类模型。

二、实验原理

使用 Python 读取数据集信息并对数据进行预处理，随后利用 Python 实现分类决策树构建，完成决策树节点定义、基尼系数计算、分类决策树生成并实现决策树可视化。

三、实验内容

本案例使用 Python 逐步实现了决策树分类模型。在实现的过程中，借助 Matplotlib 和 Networkx 等工具对决策树进行了可视化。

1、读取数据并进行探索性分析

- (1) 在 Sklearn 中可以使用 `datasets.load_iris` 方法直接加载鸢尾花数据集；

```
from sklearn.datasets import load_iris
```

```
import pandas as pd
```

- (2) 读取鸢尾花数据；

```
iris = load_iris()
```

```
iris_df = pd.DataFrame(data=iris.data,columns=iris.feature_names)
```

```
iris_df["target"] = iris.target
```

```
iris_df.head()
```

```
iris_df.columns = ["sepal_len","sepal_width","petal_len","petal_width","target"]
```

```
X_iris = iris_df.iloc[:, :-1]
```

```
y_iris = iris_df["target"]
```

- (3) 展示表格前 5 行；

2、实现分类决策树

2.1 定义决策树节点

- (1) 创建一个表示决策树节点的类 `TreeNode`;

```
class TresNode
```

```
def __init__(self,x_pos,y_pos,layer,class_labels=[0,1,2]):
```

- (2) 决策树节点需要存储的最重要的信息为当前节点代表的特征 `f` 和切分点 `v`;

```
self.f = None #当前节点的切分特征
```

```
self.v = None #当前节点的切分点
```

(3) 其次如果节点是非叶子节点，还需要存储左儿子节点 `left` 和右儿子节点 `right`;

```
self.left = None #左儿子节点
```

```
self.right = None #右儿子节点
```

```
self.pos = (x_pos,y_pos) # 节点坐标，可视化用
```

(4) 为了便于决策树进行预测，在每个节点中还要保存当前节点样本的分布 `label_dist`，它将在决策树生成时提供;

```
self.label_dist = None #当前节点样本的类分布
```

```
self.layer = layer
```

```
self.class_labels = class_labels
```

(5) 在本案例的后续，还要把决策树绘制出来，为了方便绘制，给每一个节点还保存了节点的坐标信息 `pos`，也将在决策树生成时计算。

```
def __str__(self): #打印节点信息，可视化时的节点标签
```

```
    if self.f != None:
```

```
        return self.f + "\n<=" + str(round(self.v,2))
```

```
    else:
```

```
        return str(self.label_dist) + "\n(" + str(np.sum(self.label_dist)) + ")"
```

上面的代码中，`__str__`函数定义调用打印函数 `print` 时输出的信息。对于非叶子节点（判断条件 `self.f != None`），打印节点的特征和切分点；对于叶子节点，打印节点训练样本的类别分布和样本数量。这些信息将在绘制决策树时作为节点的标签信息。

2.2 基尼系数计算

Gini 系数是一种度量落在决策树中某一个节点的样本分布不纯度的指标。假设数据集中样本一共有 C 个类别，在节点 t 中第 c 类样本的比例为 $p_{t,c}$ ，则节点 t 的不纯度为

$$Gini(t) = 1 - \sum_{c=1}^C p_{t,c}^2$$

假设样本的类别为正负两类，假设某个节点一共有 10 个样本，包括 5 个正样本和 5 个负样本，则基尼系数为

$$1 - \left(\frac{5}{10}\right)^2 - \left(\frac{5}{10}\right)^2 = \frac{1}{2}$$

假设另一个节点有 20 个样本，包含 15 个正样本和 5 个负样本，则基尼系数为

$$1 - \left(\frac{15}{20}\right)^2 - \left(\frac{15}{20}\right)^2 = \frac{3}{8}$$

实现一个不纯度计算方法 `gini` ,它的输入是当前节点样本的标签序列 `y` 。`y` 是 `Pandas` 的 `Series` 结构。使用 `value_counts` 方法计算每一类样本的数量，然后借助 `np.square` 和 `sum` 函数计算其平方和。

$$1 - \text{np.square}(y.\text{value_counts()} / \text{len}(y)).\text{sum}()$$

2.2 分类决策树生成

(1) 实现 `generate` 函数：输入数据子集，输入创建好的节点，其中 `x` 和 `y` 分别为训练集特征部分和标签部分，是该函数的主要输入，其余的输入参数为一些辅助函数，其主要含义如下表所示：

输入参数	含义说明
<code>x</code>	数据子集特征部分，格式为 <code>DataFrame</code>
<code>y</code>	数据子集特征部分，格式为 <code>Series</code>
<code>x_pos</code>	节点 <code>x</code> 轴坐标
<code>y_pos</code>	节点 <code>y</code> 轴坐标
<code>nodes</code>	用户保存决策树中所有节点的列表
<code>min_leaf_samples</code>	叶子节点最少样本数量
<code>max_depth</code>	决策树最大深度
<code>layer</code>	当前节点的层数，等于该节点父节点层数加 1
<code>class_labels</code>	样本的标签集合，格式为 <code>Series</code>

决策树生成函数 `generate` 的主要流程为：

① 创建当前节点 `current_node`，计算其样本分布 `label_dist`

```
def generate(X,y,x_pos,y_pos,nodes,min_leaf_samples,max_depth,layer,class_labels):
```

```
    current_node = TreeNode(x_pos,y_pos,layer,class_labels)#创建节点对象
```

```
    current_node.label_dist = [len(y[y==v]) for v in class_labels] #当前节点类样本分布
```

```
    nodes.append(current_node)
```

② if 当前样本数量太少，决策树层数超过最大深度或者节点的基尼系数小于某个阈值节点不再分裂，返回 current_node

```
if(len(X) < min_leaf_samples or gini(y) < 0.1 or layer > max_depth): #判断是否需要生成子节点
```

```
    return current_node
```

```
    max_gini,best_f,best_v = 0,None,None
```

③ 特征遍历、取值遍历

```
for f in X.columns: #特征遍历
```

```
    for v in X[f].unique(): #取值遍历
```

```
        y1,y2 = y[X[f] <= v],y[X[f] > v]
```

```
        if (len(y1) >= min_leaf_samples and len(y2) >= min_leaf_samples):
```

```
            imp_descent = _____ #****计算不纯度变化****#
```

```
            if imp_descent > max_gini:
```

```
                max_gini,best_f,best_v = imp_descent,f,v
```

```
current_node.f,current_node.v = best_f,best_v
```

```
if(current_node.f != None):
```

```
    current_node.left = generate(X[X[best_f] <= best_v],y[X[best_f] <= best_v],x_pos-(2**(max_depth-layer)),y_pos
```

```
-1,nodes,min_leaf_samples,max_depth,layer + 1,class_labels)
```

```
    current_node.right = generate(X[X[best_f] > best_v],y[X[best_f] > best_v],x_pos+(2**(max_depth-layer)),y_pos
```

```
-1,nodes,min_leaf_samples,max_depth,layer + 1,class_labels)
```

```
    return current_node
```

④ 封装函数：为了后续使用方便，创建一个封装函数 decision_tree_classifier，其输入为训练数据，叶子节点最小样本数和树的最大深度。返回树的根，节点集合。

```
def decision_tree_classifier(X,y,min_leaf_samples,max_depth):
```

```
    这里定义叶子节点空列表
```

```
    root = generate(X, y, 0, 0, nodes, min_leaf_samples = min_leaf_samples, max_depth=max_depth, layer=1, class_labels = y.unique())
```

```
    return root, 上面定义的节点
```

2.3 使用 Networkx 将决策树可视化

Networkx 是一个 Python 中流行的网络分析工具，实现了网络的可视化函数。

我们将使用该工具的 draw_networkx 函数对生成的决策树进行可视化。

draw_networkx 函数需要输入 Networkx 中的网络对象，节点坐标，节点颜

色，节点大小等参数,列举如下表所示。

参数	含义说明
G	Networkx 网络对象
pos	指定节点坐标的字典对象，键为节点，值为坐标(x,y)
ax	Matplotlib Axes 对象，将网络绘制在该子图上
node_shape	节点形状，常见的有圆形("o")，方形("s")，上三角形("^")
font_color	节点标签字体颜色
node_size	节点大小
node_color	节点颜色

编写一个函数，将训练得到的决策树转换成 Networkx 中的网络对象 G。

- (1) 定义函数：`def get_networkx_graph(G,root):`
- (2) 判断树左侧节点是否为空，非空则在当前节点和左儿子节点之间建立一条边，加入 G，对左侧儿子节点执行同样操作；
- (3) 判断树右侧节点是否为空，非空则在当前节点和右儿子节点之间建立一条边，加入 G，对右侧儿子节点执行同样操作；
- (4) 在决策树生成函数中，已经计算了节点的坐标。现在我们实现一个函数，输入节点集合，返回其位置布局字典对象。

```
def get_tree_pos(G):  
    pos = {}  
    for node in G.nodes:  
        pos[node] = node.pos  
    return pos
```

- (5) 在决策树中，如果是叶子节点，则根据其预测类别显示不同颜色。如果是非叶子节点，则显示灰色。

```
def get_node_color(G):  
    color_dict = []  
    for node in G.nodes:  
        if node.f == None: #叶子节点  
            label = np.argmax(node.label_dist)  
            if label%3 == 0:  
                color_dict.append("#007979") #深绿色
```

```

elif label%3 == 1:
    color_dict.append("#E4007F") #洋红色
else:
    color_dict.append("blue")
else:
    color_dict.append("gray")
return color_dict

```

(6) 在鸢尾花数据集上训练决策树，然后将学习到的决策树绘制出来，得到：

